

Đồ thị: mô hình hoá và duyệt (Graphs: Modeling and Traversal)

PHỤ THUỘC B/07 cho ba loại kho chờ — chương này là nơi chúng sinh ra ba thuật toán; B/11 cho DSU; C/15 cho chứng minh tham lam của Dijkstra và Kruskal; C/16 cho quan hệ giữa quy hoạch động và đường đi ngắn nhất.

TÓM TẮT

Kỹ năng quan trọng nhất của chương này không phải thuộc thuật toán mà là *mô hình hoá*: nhận ra rằng bài toán trước mặt, dù nói về gì, thực chất là bài toán về đỉnh và cạnh. Khi làm được điều đó, hàng chục bài trông khác nhau sụp xuống thành vài bài đã có lời giải. Chương đi qua hai phép duyệt nền tảng và những thứ chúng để lộ ra — thứ tự tô-pô, thành phần liên thông mạnh, cầu và khớp — rồi tới bốn thuật toán đường đi ngắn nhất, mỗi cái ứng với một giả thiết khác nhau về trọng số. Phần cuối là các mẹo mô hình hoá: đồ thị trạng thái, đồ thị nhiều tầng, tách đỉnh. Nếu chỉ nhớ một điều: khi bí, hãy hỏi “cái gì là đỉnh, cái gì là cạnh” — câu hỏi đó giải được nhiều bài hơn bất kỳ thuật toán riêng lẻ nào.

KIẾN THỨC CẦN CÓ TRƯỚC

- **đồ thị** — tập đỉnh và tập cạnh nối chúng; có hướng nếu cạnh có chiều, có trọng số nếu cạnh mang một giá trị.
- **danh sách kề** — cách lưu phổ biến nhất: với mỗi đỉnh, một danh sách các đỉnh kề. Bộ nhớ $O(n+m)$, hợp với đồ thị thưa.
- **duyet rộng (BFS)** — duyệt theo lớp khoảng cách bằng hàng đợi; cho đường đi ngắn nhất trên đồ thị không trọng số.
- **duyet sâu (DFS)** — đi sâu hết một nhánh rồi quay lại, dùng ngăn xếp hoặc đệ quy; thời điểm vào/ra của mỗi đỉnh mang thông tin cấu trúc.
- **thứ tự tô-pô** — cách xếp các đỉnh của một đồ thị có hướng không chu trình sao cho mọi cạnh đi từ trái sang phải.
- **thành phần liên thông mạnh** — tập đỉnh mà từ đỉnh nào cũng tới được mọi đỉnh khác trong tập; nén chúng lại cho một đồ thị không chu trình.

CÂU HỎI MỞ ĐẦU

- Q1. Nhiều bài toán không hề nhắc tới “đồ thị” vẫn giải bằng đồ thị. Dấu hiệu nào cho biết nên phát biểu lại bài toán theo đỉnh và cạnh?
- Q2. Duyệt rộng và duyệt sâu chỉ khác nhau ở một dòng code. Vì sao chúng để lộ ra những thông tin hoàn toàn khác nhau?
- Q3. Dijkstra sai với cạnh trọng số âm. Chỗ nào trong lập luận của nó dùng tới giả thiết không âm, và Bellman–Ford làm khác thế nào?
- Q4. Vì sao “đường đi ngắn nhất” và “quy hoạch động” lại là hai cách nói về cùng một thứ?
- Q5. Khi đồ thị có tới 10^5 đỉnh và cần trả lời hàng trăm nghìn truy vấn về tổ tiên chung trên cây, làm sao trả lời mỗi truy vấn trong $O(\log n)$?

Có một khoảnh khắc rất đặc trưng khi luyện thuật toán: bạn đọc một đề bài nói về những người bạn trao đổi quà, hoặc về các công việc phụ thuộc nhau, hoặc về các trạng thái của một trò xếp hình — và đột nhiên nhận ra đó là bài toán tìm chu trình, hoặc sắp thứ tự tô-pô, hoặc đường đi ngắn nhất. Sau khoảnh khắc đó, bài toán gần như đã xong: phần còn lại là gọi một thuật toán bạn đã biết.

Chương này viết cho chính khoảnh khắc đó. Các thuật toán đồ thị đều đã được chuẩn hoá và bạn tra được ở bất cứ đâu; cái không tra được là kỹ năng nhìn ra rằng bài toán của mình *là* một bài đồ thị. Vì vậy tôi sẽ trình bày thuật toán ở mức đủ dùng, và dành trọng lượng cho câu hỏi mô hình hoá — cái gì là đỉnh, cái gì là cạnh, và cạnh mang trọng số gì.

1.1 Mô hình hoá: cái gì là đỉnh, cái gì là cạnh

Ba dấu hiệu cho thấy bài toán nên được phát biểu lại thành đồ thị.

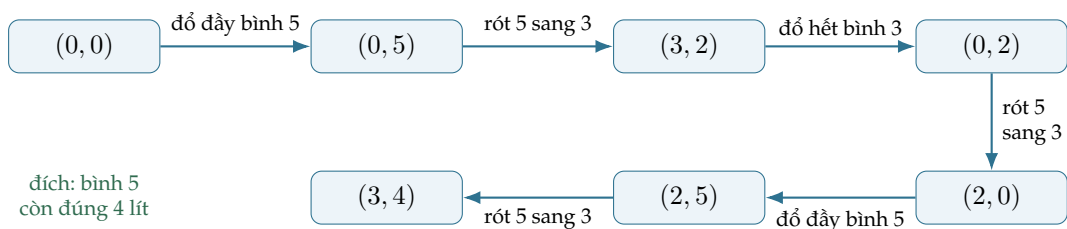
Dấu hiệu thứ nhất — quan hệ đôi một. Đề nói về những cặp có quan hệ với nhau: hai người quen nhau, hai thành phố có đường nối, hai công việc xung khắc. Đỉnh là đối tượng, cạnh là quan hệ. Đây là trường hợp hiển nhiên nhất.

Dấu hiệu thứ hai — phụ thuộc và thứ tự. Đề nói “việc A phải làm trước việc B”. Đỉnh là việc, cạnh có hướng là ràng buộc; câu hỏi “có xếp được thứ tự không” trở thành “đồ thị có chu trình không”.

Dấu hiệu thứ ba — và đây là dấu hiệu đáng giá nhất — trạng thái và bước chuyển. Đề mô tả một hệ có thể ở nhiều tình huống, và có các thao tác đưa từ tình huống này sang tình huống khác. Lúc đó *đỉnh là trạng thái, cạnh là một thao tác hợp lệ*, và câu hỏi “ít thao tác nhất để tới đích” chính là đường đi ngắn nhất.

Dấu hiệu thứ ba mở ra rất nhiều bài mà nhìn bề ngoài không có đồ thị nào: trò xếp hình, các bài đổ nước giữa các bình, bài biến chuỗi này thành chuỗi kia bằng các phép biến đổi, bài robot di chuyển trên lưới có luật. Điều cần cẩn thận là *kích thước không gian trạng thái*: nếu số trạng thái là 10^6 thì duyệt rộng chạy tốt; nếu là 10^{15} thì phải tìm cấu trúc khác hoặc dùng tìm kiếm có định hướng.

Hình 1.1 ghi rõ từng thao tác; kiểm từng cạnh là cách kiểm mô hình trước khi viết BFS.



Hình 1.1. Một đường đi hợp lệ gồm sáu thao tác từ hai bình rỗng tới 4 lít trong bình 5 lít. Trạng thái là (lượng trong bình 3, lượng trong bình 5). Hình chỉ vẽ một đường đi, không phải toàn bộ đồ thị BFS.

CẦN NHỚ

Ba câu hỏi khi nghi ngờ bài là bài đồ thị: (i) đỉnh là gì — đối tượng hay *trạng thái*; (ii) cạnh là gì, và có hướng không; (iii) cạnh có trọng số gì — không trọng số, trọng số 0/1, trọng số dương, hay có thể âm. Câu thứ ba quyết định thuật toán, và bảng ở mục 4 là bản đồ.

1.2 Hai phép duyệt và những gì chúng để lộ

Duyệt rộng và duyệt sâu chỉ khác nhau ở kho chờ — hàng đợi hay ngăn xếp, đúng như chương B/07 đã nói — nhưng thông tin chúng cho lại rất khác.

Duyệt rộng xử lý các đỉnh theo lớp khoảng cách tăng dần. Hệ quả trực tiếp: nó cho *đường đi ngắn nhất trên đồ thị không trọng số*, và khoảng cách tìm được là chính xác chứ không phải xấp xỉ. Hai biến thể đáng thuộc: *duyet rộng nhiều nguồn* — đẩy tất cả các nguồn vào hàng đợi lúc đầu, cho “khoảng cách tới nguồn gần nhất” trong một lượt; và *duyet rộng 0-1* — khi trọng số chỉ là 0 hoặc 1, dùng deque và đẩy vào đầu khi cạnh trọng số 0, cho $O(n + m)$ thay vì $O(m \log n)$ của Dijkstra.

Duyệt sâu không cho khoảng cách, nhưng nó cho một thứ khác có giá trị hơn: *cấu trúc*. Khi ghi lại thời điểm vào và thời điểm ra của mỗi đỉnh, ta được một bộ khung để đọc ra rất nhiều tính chất. Đây chính là đóng góp khái niệm lớn của Tarjan năm 1972[1]: biến duyệt sâu từ “một cách đi lòng vòng” thành một công cụ phân tích có hệ thống.

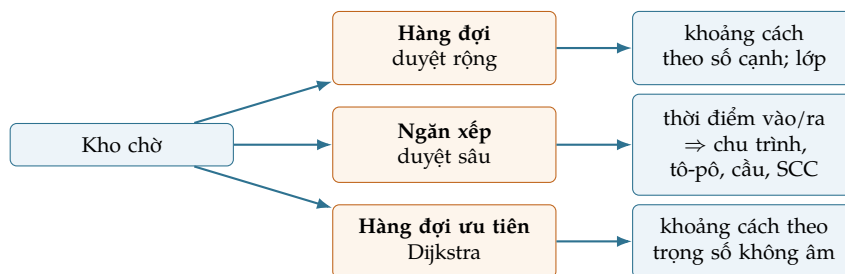
Bốn thứ mà thời điểm vào/ra của duyệt sâu để lộ ra:

Chu trình. Trong đồ thị có hướng, có chu trình khi và chỉ khi tồn tại cạnh đi ngược về một đỉnh đang nằm trên ngăn xếp hiện tại.

Thứ tự tô-pô. Trên đồ thị có hướng không chu trình (DAG), xếp các đỉnh theo thời điểm *ra* giảm dần cho một thứ tự tô-pô hợp lệ. Nếu phát hiện chu trình thì không có thứ tự tô-pô.

Cầu và khớp. Một cạnh là cầu khi cây con bên dưới nó không có đường ngược nào lên phía trên — điều kiện này viết được bằng thời điểm vào và một giá trị “lên cao nhất có thể”.

Thành phần liên thông mạnh. Cả hai thuật toán chuẩn — Kosaraju với hai lượt duyệt, Tarjan với một lượt — đều dựa trên cùng bộ thông tin đó. Sau khi nén mỗi thành phần thành một đỉnh, ta được đồ thị không chu trình, và mọi kỹ thuật quy hoạch động trên đồ thị không chu trình dùng lại được.



Hình 1.2. Cùng một khung “lấy ra, xử lý, đẩy vào”, đôi kho chờ thì được ba thuật toán với ba loại kết quả khác nhau. Nhận ra điều này giúp nhớ ít hơn và suy được nhiều hơn — và nó cũng giải thích ngay vì sao Dijkstra cần trọng số không âm (B/07).

1.3 Bốn thuật toán đường đi ngắn nhất, bốn giả thiết

Chọn thuật toán đường đi ngắn nhất không phải chuyện sở thích: mỗi thuật toán ứng với một giả thiết về trọng số, và dùng sai giả thiết cho kết quả sai chứ không phải chậm.

Không trọng số → duyệt rộng, $O(n + m)$. Đơn giản nhất, nhanh nhất; đừng dùng Dijkstra ở đây.

Trọng số 0 hoặc 1 → duyệt rộng 0-1 với deque, $O(n + m)$.

Trọng số không âm → Dijkstra với hàng đợi ưu tiên, $O(m \log n)$. Nó là thuật toán tham lam: chốt đỉnh có nhãn nhỏ nhất và không xét lại. Chứng minh dùng giả thiết không âm đúng một lần — ở chỗ khẳng định rằng một đường đi vòng qua các đỉnh chưa chốt không thể ngắn hơn — và đó chính là chỗ nó hỏng khi có cạnh âm.

Có trọng số âm $\rightarrow$ Bellman–Ford, $O(nm)$. Nó không chốt gì cả: nó thư giãn mọi cạnh $n - 1$ lần, và ý tưởng là “đường đi ngắn nhất có tối đa $n - 1$ cạnh, nên sau $n - 1$ vòng mọi nhãn đã ổn định”. Lợi thế kèm theo: nếu ở vòng thứ n vẫn còn cạnh thư giãn được thì đồ thị có chu trình âm — một thứ mà Dijkstra không phát hiện được.

Mọi cặp đỉnh $\rightarrow$ Floyd–Warshall, $O(n^3)$. Đáng chú ý vì cách nghĩ của nó là quy hoạch động thuần túy: $f[k][i][j]$ là đường ngắn nhất từ i tới j chỉ dùng các đỉnh trung gian trong k đỉnh đầu tiên. Ba vòng lặp lồng nhau, và thứ tự của chúng không hoán đổi được — vòng k phải nằm ngoài cùng, một lỗi rất phổ biến.

Ngoài bốn cái trên còn hai công cụ nên biết. A^* là Dijkstra cộng một hàm ước lượng khoảng cách còn lại; nếu hàm đó không bao giờ đánh giá quá thực tế thì kết quả vẫn tối ưu, còn nếu đánh giá tốt thì số đỉnh phải xét giảm rất nhiều. Đây là thuật toán chuẩn cho tìm đường của robot và trò chơi. Johnson biến đổi trọng số để loại bỏ số âm rồi chạy Dijkstra từ mọi đỉnh — hợp cho đồ thị thưa có cạnh âm.

Giả thiết trọng số	Thuật toán	Chi phí	Điều cần nhớ
Không trọng số	Duyệt rộng	$O(n + m)$	Nhanh nhất; nhiều nguồn cùng lúc rất tiện
Chỉ 0 và 1	Duyệt rộng 0–1	$O(n + m)$	Deque: cạnh 0 đẩy vào đầu, cạnh 1 vào cuối
Không âm	Dijkstra	$O(m \log n)$	Nhớ bỏ qua bản ghi lỗi thời khi lấy ra
Có thể âm	Bellman–Ford	$O(nm)$	Phát hiện được chu trình âm ở vòng thứ n
Mọi cặp, n nhỏ	Floyd–Warshall	$O(n^3)$	Vòng k bắt buộc ở ngoài cùng
Có ước lượng tốt	A^*	phụ thuộc heuristic	Heuristic không được đánh giá quá thực tế

1.4 Cây: các kỹ thuật riêng

Cây là đồ thị liên thông không chu trình, và chính vì thiếu chu trình mà nhiều bài trở nên dễ hơn hẳn. Ba kỹ thuật đáng thuộc.

Nhảy nhị phân cho tổ tiên chung gần nhất. Tiền xử lý $up[v][k] =$ tổ tiên thứ 2^k của v , tốn $O(n \log n)$; sau đó mỗi truy vấn tổ tiên chung gần nhất trả lời trong $O(\log n)$ bằng cách nâng hai đỉnh về cùng độ sâu rồi nhảy đồng thời. Kỹ thuật này còn dùng để tính nhanh cực trị hoặc tổng trên đường đi giữa hai đỉnh — chỉ cần lưu thêm thông tin tổng hợp cùng với bảng nhảy.

Dãy Euler. Duyệt sâu và ghi lại đỉnh mỗi khi đi qua, ta được một dãy mà bài toán tổ tiên chung trở thành bài toán cực tiểu trên đoạn — tức là chuyển hẳn sang công cụ của chương B/10. Đây là một trong những phép quy dẫn đẹp nhất giữa hai miền, và nó đi cả hai chiều: mọi bài cực tiểu trên đoạn cũng dựng được thành bài tổ tiên chung trên cây Cartesian.

Phân rã theo cạnh nặng. Chia cây thành các chuỗi dọc sao cho mọi đường đi từ gốc xuống lá chỉ cắt qua $O(\log n)$ chuỗi; nhờ đó truy vấn và cập nhật trên đường đi giữa hai đỉnh quy về $O(\log n)$ truy vấn đoạn trên các chuỗi. Xếp vào loại tham khảo nhưng rất mạnh khi cần.

1.5 Ba mẹo mô hình hoá đáng giá nhất

Ba mẹo dưới đây biến những bài trông không giải được thành bài quen thuộc.

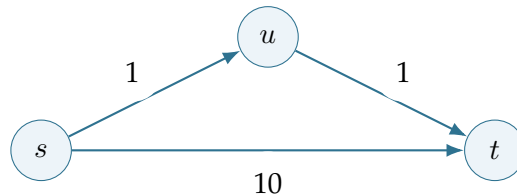
Đồ thị trạng thái. Đã nói ở mục 1, nhưng đáng nhắc lại kèm ví dụ: bài “có hai bình dung tích 3 và 5 lít, đong ra đúng 4 lít” có đỉnh là cặp (lượng nước bình 1, lượng nước bình 2) và cạnh là các thao tác đổ. Bài trở thành duyệt rộng trên vài chục đỉnh.

Đồ thị nhiều tầng. Khi bài cho phép một số hành động đặc biệt với số lần hạn chế — “được đi miễn phí tối đa k cạnh” — hãy nhân bản đồ thị thành $k + 1$ tầng, mỗi tầng ứng với số lần đã dùng, và cho cạnh đặc biệt nối tầng i sang tầng $i + 1$. Đường đi ngắn nhất trên đồ thị mở rộng chính là đáp án. Mẹo này là ứng dụng trực tiếp của nguyên tắc “trạng thái phải đủ” ở chương C/16: số lần đã dùng là một phần của trạng thái.

Tách đỉnh. Khi ràng buộc nằm ở *đỉnh* chứ không ở *cạnh* — “mỗi thành phố chỉ đi qua một lần”, “mỗi máy chỉ xử lý một việc” — hãy tách mỗi đỉnh thành hai nửa nối bởi một cạnh mang đúng ràng buộc đó. Kỹ thuật này đặc biệt quan trọng trong các bài luồng ở chương C/18.

1.6 Phản ví dụ ba đỉnh để chọn đúng đường đi ngắn nhất

Hình 1.3 có đường một cạnh $s \rightarrow t$ giá 10 và đường hai cạnh $s \rightarrow u \rightarrow t$ giá 2. BFS tối thiểu số cạnh sẽ chọn đường giá 10; Dijkstra tối thiểu tổng trọng số chọn đường giá 2. Vì vậy thay queue bằng heap không đủ: phải thay cả luật cập nhật khoảng cách bằng phép nối lỏng.



Hình 1.3. Ít cạnh nhất không đồng nghĩa tổng trọng số nhỏ nhất. Cả ba cạnh có hướng và trọng số không âm; Dijkstra tìm khoảng cách 2 tới t .

Khi dùng heap có phần tử cũ, mỗi lần lấy (d, u) , bỏ qua nếu $d \neq \text{dist}[u]$. Chỉ dừng sớm khi lấy đích ra với khoảng cách còn hợp lệ, không dừng lúc vừa nhìn thấy cạnh tới đích. Muốn thấy vì sao cấm cạnh âm, đổi $s \rightarrow t$ thành 2, $s \rightarrow u$ thành 5 và $u \rightarrow t$ thành -10 : chốt t ở 2 sẽ bỏ lỡ đường giá -5 . Đồ thị này là DAG nên vẫn giải được bằng thứ tự tô-pô.

Bảng 1.1. Bổ sung hai trường hợp dễ bị bỏ sót khi chọn đường đi ngắn nhất. n là số đỉnh, m là số cạnh, dùng danh sách kề.

Thuật toán	Giả thiết	Chi phí	Lỗi cần tránh
BFS	Mọi cạnh cùng giá dương	$O(n + m)$	Tối thiểu số cạnh trên đồ thị có giá khác nhau
0–1 BFS	Giá cạnh chỉ 0 hoặc 1	$O(n + m)$	Dùng visited để chặn mọi lần cải thiện
DP trên DAG	Có hướng, không chu trình; cho phép cạnh âm	$O(n + m)$	Dùng thứ tự DFS khi chưa loại chu trình
Dijkstra + heap nhị phân	Giá cạnh không âm	$O((n + m) \log n)$ trên đồ thị đơn	Chốt đỉnh lúc thêm vào heap
Bellman–Ford	Cho phép cạnh âm	$O(nm)$	Bỏ qua chu trình âm tới được từ nguồn

Bảng 1.1 tách giả thiết về trọng số khỏi giả thiết về cấu trúc. Nếu chu trình âm tới được từ nguồn và đi tiếp được tới đích, không tồn tại khoảng cách hữu hạn nhỏ nhất tới đích.

1.7 Dijkstra có phải là tận cùng không: rào cản sắp xếp

Bảng trên ghi Dijkstra là $O(m \log n)$ và dừng ở đó, như mọi giáo trình. Nhưng cái log ấy đến từ đâu, và có bắt buộc không? Câu hỏi này có một lịch sử dài và một câu trả lời rất mới, đáng kể lại vì nó cho thấy cách một “sự thật hiển nhiên” bị lật.

Nguồn của log là hàng đợi ưu tiên: Dijkstra rút các đỉnh ra *theo đúng thứ tự khoảng cách tăng dần*, nghĩa là chạy xong thì nó đã sắp xếp n đỉnh theo khoảng cách. Mà sắp xếp n số bằng so sánh cần $\Omega(n \log n)$ (D/24). Vậy mọi thuật toán làm đúng như Dijkstra — chốt đỉnh theo thứ tự — đều bị chặn dưới bởi chi phí sắp xếp. Người ta gọi đó là *rào cản sắp xếp*, và với hàng đợi Fibonacci[2] thì cận $O(m + n \log n)$ chính là chạm đúng rào cản ấy.

Cách vượt rào thứ nhất là bỏ mô hình so sánh. Nếu trọng số là số nguyên và máy được phép làm số học trên chúng, cận dưới sắp xếp không còn áp dụng — đúng như sắp xếp đếm không mâu thuẫn với cận $n \log n$. Thorup[3] khai thác đúng chỗ đó và cho thuật toán *tuyến tính* $O(m)$ cho đồ thị *vô hướng* với trọng số nguyên dương, bằng cách xây một cấu trúc phân tầng theo bit và chấp nhận xử lý các đỉnh “theo nhóm không cần đúng thứ tự”. Chi tiết đáng nhớ là chỗ giả thiết được dùng: vô hướng và nguyên, cả hai đều thiết yếu.

Cách vượt rào thứ hai mới xuất hiện gần đây và mạnh hơn về mặt khái niệm, vì nó *không* bỏ mô hình so sánh. Duan, Mao, Mao, Shu và Yin[4] đưa ra thuật toán tất định $O(m \log^{2/3} n)$ cho đồ thị *có hướng* với trọng số thực không âm, trong mô hình chỉ có so sánh và cộng — tức là phá vỡ cận $O(m + n \log n)$ của Dijkstra trên đồ thị thưa và chứng minh rằng Dijkstra *không* tối ưu cho bài toán đường đi ngắn nhất. Ý tưởng cốt lõi là làm ít hơn cái Dijkstra làm: thuật toán tránh việc sắp xếp đầy đủ mọi đỉnh theo khoảng cách, và chỉ giữ đủ thứ tự để các nhãn vẫn đúng. Bài được trao giải bài báo xuất sắc tại STOC 2025.

Đừng cài các thuật toán này. Giá trị của mục này nằm ở ba câu hỏi mà nó dạy bạn tự đặt cho *bất kỳ* thuật toán quen thuộc nào. Một, thuật toán này có đang làm nhiều hơn việc đề bài yêu cầu không — Dijkstra sắp xếp toàn bộ đỉnh trong khi đề chỉ hỏi khoảng cách. Hai, cận dưới mà tôi vừa viện dẫn thuộc mô hình nào, và bài toán của tôi có thật sự nằm trong mô hình đó không. Ba, khi một cận đứng yên hàng chục năm, đó là dấu hiệu của một định lý hay chỉ là dấu hiệu chưa ai thử đúng chỗ? Với đường đi ngắn nhất, câu trả lời hoá ra là về sau — sau sáu mươi sáu năm.

1.8 Phản biện, nhược điểm và rủi ro triển khai

Đồ thị là ngôn ngữ mô hình hoá mạnh nhất trong thuật toán, nhưng sự phổ quát của nó thường khiến kỹ sư chủ quan trước các cái giá về phần cứng và các chế độ hỏng thảm hoạ trong môi trường thực tế.

Phản biện: sự lạm dụng của tư duy rời rạc hoá. Mô hình hoá thành đồ thị là con dao hai lưỡi. Trong các bài toán thực tế của robot và xe tự hành (như dẫn đường trong không gian liên tục), việc băm không gian thành lưới rồi chạy Dijkstra hay A^* thường sinh ra các đường đi zíc-zắc gây khúc (discretization artifacts), không khả thi về mặt động học trơn của xe. Khi số bậc tự do tăng lên (như cánh tay robot 6–7 khớp), không gian trạng thái bùng nổ theo hàm mũ khiến mọi đồ thị rời rạc đều cạn kiệt bộ nhớ; lúc đó các thuật toán tối ưu liên tục (như TrajOpt, CHOMP) hoặc cây ngẫu nhiên hoá (RRT*, PRM) vượt trội hơn hẳn. Ngoài ra, với A^* , niềm tin rằng “ A^* luôn nhanh hơn Dijkstra” là ảo tưởng: nếu hàm heuristic $h(u)$ tính toán tốn kém hoặc không định hướng tốt ($h \equiv 0$), A^* chạy chậm hơn Dijkstra vì gánh thêm chi phí tính toán cho từng đỉnh; còn nếu $h(u)$ vô tình đánh giá cao hơn chi phí thật (vi phạm tính chấp nhận được / admissibility), thuật toán sẽ âm thầm trả về đường đi không tối ưu mà không hề báo lỗi.

Nhược điểm cốt lõi về tài nguyên và phân cấp bộ nhớ:

- *Thảm hoạ trượt cache của danh sách kề (Pointer Chasing)*: Cách cài đặt phổ biến `vector<int> adj[N]` tạo ra N mảng con nằm phân tán rải rác trên heap. Khi duyệt đồ thị có hàng triệu đỉnh, CPU liên tục phải nhảy con trỏ ngẫu nhiên khắp bộ nhớ, gây trượt cache L1/L2 liên tục và làm chậm chương trình từ 3 tới 5 lần so với cấu trúc mảng nén liên tục CSR (Compressed Sparse Row).
- *Phình to bộ nhớ trong Dijkstra dùng hàng đợi ưu tiên chuẩn*: Thư viện chuẩn C++ (`std::priority_queue`) không hỗ trợ thao tác giảm khoá (`decrease-key`). Để cập nhật nhãn, ta buộc phải đẩy một cặp (d, u) mới vào heap, khiến số phần tử trong heap có thể lên tới m thay vì n . Trên đồ thị dày ($m \approx 10^7$), heap ngốn hàng trăm megabyte RAM và làm tăng chi phí mỗi lần rút đỉnh lên $O(\log m)$.
- *Sự thoái hoá của SPFA (Shortest Path Faster Algorithm)*: SPFA (bản cải tiến dùng hàng đợi của Bellman–Ford) chạy rất nhanh trên đồ thị ngẫu nhiên (trung bình $O(km)$), khiến nhiều người nhầm tưởng nó thay thế được Dijkstra. Nhưng trong trường hợp xấu nhất do đối thủ cố ý dựng (đồ thị dạng lưới hoặc chuỗi lồng nhau), SPFA thoái hoá về đúng $O(nm)$, thậm chí bị treo hoàn toàn khi các tối ưu như SLF/LLL gặp dữ liệu đối kháng.

Rủi ro vận hành và các lỗi im lặng nguy hiểm:

- *Tràn ngăn xếp đệ quy (Stack Overflow) trong DFS*: DFS đệ quy ngây thơ là nguyên nhân số một gây lỗi sập chương trình đột ngột (Segmentation Fault). Trên các hệ thống Linux với giới hạn ngăn xếp mặc định 8 MB, một cây suy biến thành đường thẳng có $n \geq 10^5$ đỉnh sẽ làm tràn ngăn xếp chỉ sau vài phần mười giây. Trong hệ thống thật, DFS trên đồ thị lớn bắt buộc phải được viết ở dạng lặp với một ngăn xếp tường minh cấp phát trên heap.
- *Tràn số nguyên trong Floyd–Warshall*: Để biểu diễn “không có đường nối”, ta khởi tạo ma trận khoảng cách bằng ∞ . Nếu chọn $\infty = 2 \cdot 10^9$ (gần `INT_MAX`), phép cộng $dist[i][k] + dist[k][j]$ sẽ tràn số học thành một số âm cực lớn, khiến thuật toán nhầm tưởng rằng giữa i và j tồn tại một đường đi âm siêu ngắn. Quy tắc: dùng hằng số an toàn như `0x3f3f3f3f` (cho kiểu 32-bit) hoặc 10^{18} (cho kiểu 64-bit).
- *Lập vô hạn trong Dijkstra trên số thực*: Khi cạnh mang trọng số thực và đồ thị xuất hiện các chu trình có chi phí xấp xỉ 0 do sai số làm tròn số thực dấu phẩy động (ví dụ sai số -10^{-16}), điều kiện kiểm tra khoảng cách có thể bị đánh lừa liên tục, đẩy thuật toán vào vòng lặp vô tận. Bắt buộc phải có ngưỡng dung sai ε (eps) khi so sánh các nhãn khoảng cách thực.
- *Bẫy đồ thị không liên thông*: Rất nhiều chương trình chỉ chạy BFS/DFS từ đỉnh 1 và ngầm giả định đã duyệt hết mọi đỉnh. Khi đồ thị bị đứt gãy thành nhiều thành phần liên thông độc lập, các đỉnh còn lại sẽ hoàn toàn bị bỏ quên, dẫn tới nghiệm sai lệch âm thầm khi tìm cầu, khớp hoặc thành phần liên thông mạnh.

CẠM BÃY

Trong Dijkstra với heap chuẩn C++, dòng kiểm tra `if (d > dist[u]) continue;` không phải là một tối ưu nhỏ để chạy nhanh hơn — nó là điều kiện sống còn để ngăn chặn thuật toán duyệt lại các đỉnh cũ và làm độ phức tạp bùng nổ theo cấp số nhân trên các đồ thị có nhiều đường đi thay thế.

1.9 Gốc rễ: bảy cây cầu, một quán cà phê, và một con robot

Lý thuyết đồ thị có một ngày sinh khá rõ ràng: năm 1736, Euler giải bài toán bảy cây cầu ở Königsberg — có đi qua mỗi cây cầu đúng một lần rồi về chỗ cũ được không. Điều làm bài báo đó thành khởi đầu của cả một ngành không phải câu trả lời (không) mà là *cách trả lời*: Euler bỏ đi toàn bộ hình dạng địa lý, giữ lại đúng quan hệ “vùng đất nào nối với vùng đất nào”, và chứng minh kết

luận chỉ từ tính chắn lẻ của số cầu tại mỗi vùng. Đó chính xác là thao tác mô hình hoá mà mục 1 nói tới, thực hiện lần đầu tiên.

Thuật toán Dijkstra có một nguồn gốc rất đời và được chính tác giả kể lại nhiều lần: năm 1956, ông đang ngồi ở một quán cà phê tại Amsterdam cùng vợ chưa cưới, và cần một bài toán trình diễn cho máy tính ARMAC sao cho khán giả không chuyên hiểu được. Ông nghĩ ra bài đường đi ngắn nhất giữa hai thành phố Hà Lan, và tìm ra thuật toán trong khoảng hai mươi phút, không giấy bút. Ông xuất bản nó ba năm sau trong một bài báo hai trang[5]. Chi tiết đáng chú ý về mặt phương pháp: ràng buộc “phải giải thích được cho người không chuyên” đã đẩy ông tới một thuật toán đơn giản — và sự đơn giản đó là lý do nó vẫn được dùng khắp nơi sau bảy mươi năm.

Duyệt sâu thì cổ hơn nhiều — nó xuất hiện trong các phương pháp giải mê cung từ thế kỷ 19 — nhưng đóng góp quyết định là của Tarjan năm 1972[1]. Ông chỉ ra rằng nếu ghi lại thời điểm vào và ra của mỗi đỉnh thì duyệt sâu trở thành một *khung phân tích*: cùng một lượt duyệt cho ra thành phần liên thông mạnh, cầu, khớp, và nhiều tính chất khác. Trước đó mỗi bài toán có một thuật toán riêng; sau đó chúng trở thành các cách đọc khác nhau của cùng một cấu trúc dữ liệu.

Cuối cùng, A^* ra đời năm 1968 tại Viện Nghiên cứu Stanford, từ một nhu cầu rất cụ thể: robot Shakey — một trong những robot di động đầu tiên có khả năng lập kế hoạch — cần tìm đường trong một môi trường thật với tài nguyên tính toán rất hạn chế. Hart, Nilsson và Raphael thêm vào Dijkstra một hàm ước lượng khoảng cách còn lại và chứng minh điều kiện để kết quả vẫn tối ưu. Đây là lý do A^* tới nay vẫn là thuật toán lập kế hoạch đường đi mặc định trong robot học và trong trò chơi.

■ Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1736 — Euler	Bảy cây cầu Königsberg: đi qua mỗi cầu đúng một lần được không?	Khai sinh lý thuyết đồ thị; và quan trọng hơn, khai sinh thao tác <i>bỏ chi tiết, giữ quan hệ</i> — chính là mô hình hoá.
1956 — Dijkstra	Cần một bài trình diễn máy tính mà khán giả không chuyên hiểu được.	Thuật toán đường đi ngắn nhất, nghĩ ra trong 20 phút không giấy bút; ràng buộc “phải dễ hiểu” dẫn tới sự đơn giản khiến nó sống lâu.
1958–62 — Bellman, Ford, Floyd, Warshall 1968 — Hart, Nilsson, Raphael (SRI)	Mạng có chi phí âm (chiết khấu, chênh lệch tỉ giá) và nhu cầu tính mọi cặp. Robot Shakey phải lập kế hoạch đường đi thật với tài nguyên rất hạn chế.	Bellman–Ford và Floyd–Warshall; cả hai là quy hoạch động thuần túy trên đồ thị. A^* : Dijkstra cộng ước lượng khoảng cách còn lại; vẫn là thuật toán mặc định trong robot học và trò chơi.
1972 — Tarjan	Mỗi tính chất của đồ thị lại có một thuật toán riêng.	Duyệt sâu với thời điểm vào/ra trở thành khung phân tích chung: SCC, cầu, khớp, tô-pô đều đọc ra từ cùng một lượt duyệt.
Thập niên 2000 — định tuyến bản đồ	Tìm đường trên mạng đường bộ hàng trăm triệu cạnh, trả lời trong mili-giây.	Các kỹ thuật tiền xử lý như phân cấp co rút; cho thấy Dijkstra thuần túy không đủ ở quy mô thật, nhưng vẫn là nền của mọi cải tiến.

■ 1.10 Liên hệ ngang

■ Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
SLAM và robot	Đồ thị tư thế: đỉnh là tư thế, cạnh là ràng buộc đo được	Đây là mô hình hoá đúng nghĩa: bài toán ước lượng trở thành bài toán trên đồ thị, và việc đóng vòng chính là thêm một cạnh. Khác với chương này: ở đó ta không tìm đường đi mà <i>tối ưu vị trí các đỉnh</i> sao cho tổng sai lệch nhỏ nhất — một bài bình phương tối thiểu trên cấu trúc đồ thị.
Lập kế hoạch đường đi	A*, lưới chiếm dụng, đồ thị khả kiến	Đúng thuật toán của mục 3, với đỉnh là ô lưới hoặc trạng thái tư thế. Bài học chuyển được: chất lượng hàm ước lượng quyết định số đỉnh phải xét, và một hàm đánh giá quá thực tế sẽ phá vỡ tính tối ưu.
Trình biên dịch	Đồ thị luồng điều khiển, đồ thị phụ thuộc, tô màu để cấp phát thanh ghi	Sắp thứ tự tô-pô dùng để xếp lịch lệnh; phát hiện chu trình dùng để tìm vòng lặp; tô màu đồ thị dùng để gán biến vào thanh ghi — một bài NP-khó được giải bằng heuristic trong mọi trình biên dịch bạn dùng.
Mạng máy tính	Giao thức định tuyến	Định tuyến theo trạng thái liên kết chính là Dijkstra chạy phân tán; định tuyến theo vector khoảng cách chính là Bellman–Ford chạy phân tán. Đây là ví dụ hiếm hoi mà hai thuật toán sách giáo khoa tương ứng đúng với hai họ giao thức thực tế.
Quản lý dự án	Sơ đồ mạng công việc, đường găng	Đường dài nhất trên đồ thị không chu trình — giải được vì không có chu trình, đúng như C/16 đã lưu ý. Đường găng là các công việc mà chậm một ngày thì cả dự án chậm một ngày.
Web và mạng xã hội	Xếp hạng trang, phát hiện cộng đồng, đường đi ngắn nhất xã hội	Xếp hạng trang là bước ngẫu nhiên trên đồ thị, tức là chuỗi Markov — nổi chương này với xác suất ở A/05. Quy mô ở đây buộc phải dùng thuật toán xấp xỉ và tính toán phân tán.
Hoá học và sinh học	Đồ thị phân tử, mạng phản ứng, mạng tương tác protein	Bài toán đẳng cấu đồ thị — hai phân tử có cùng cấu trúc không — là một trong số ít bài toán chưa được xếp dứt khoát vào P hay NP-đầy đủ (D/23).

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Ba dấu hiệu. Một, đề nói về quan hệ đôi một giữa các đối tượng. Hai, đề nói về ràng buộc thứ tự hoặc phụ thuộc — lúc đó câu hỏi “có xếp được không” chính là “có chu trình không”. Ba, và đáng giá nhất, đề mô tả một hệ có nhiều *trạng thái* và các thao tác chuyển giữa chúng; lúc đó đỉnh là trạng thái, cạnh là thao tác, và “ít bước nhất” là đường đi ngắn nhất. Dấu hiệu thứ ba mở ra các bài không nhắc gì tới đồ thị: trò xếp hình, đong nước, biến đổi chuỗi.
- A2. Vì thứ tự xử lý quyết định thông tin được đảm bảo. Hàng đợi cho thứ tự theo lớp khoảng cách, nên lần đầu chạm tới một đỉnh là đường ngắn nhất — một tính chất mạnh và trực tiếp. Ngăn xếp cho thứ tự lồng nhau: một đỉnh chỉ ra khỏi ngăn xếp sau khi mọi thứ dưới nó đã xong, nên khoảng thời gian vào/ra của các đỉnh có quan hệ bao nhau đúng theo cấu trúc cây duyệt. Chính tính bao nhau đó là thứ cho ra thứ tự tô-pô, cầu và thành phần liên thông mạnh.
- A3. Dijkstra dùng giả thiết không âm ở đúng một chỗ: khi chốt đỉnh có nhãn nhỏ nhất, nó khẳng định rằng không có đường nào tới đỉnh đó rẻ hơn, vì mọi đường khác phải đi qua một đỉnh chưa chốt có nhãn lớn hơn hoặc bằng, và các cạnh sau đó *chỉ làm tăng* chi phí. Với cạnh âm, các cạnh sau có thể làm giảm chi phí nên lập luận sụp. Bellman–Ford không chốt gì cả: nó thư giãn mọi cạnh $n - 1$ lần, dựa trên nhận xét đường đi ngắn nhất có tối đa $n - 1$ cạnh — và nếu vòng thứ n vẫn cải thiện được thì tồn tại chu trình âm.
- A4. Vì mọi bài quy hoạch động đều là bài đường đi ngắn nhất trên *đồ thị trạng thái*: đỉnh là trạng thái, cạnh là chuyển, trọng số là chi phí của chuyển đó, và thứ tự tính hợp lệ tồn tại đúng khi đồ thị không có chu trình. Ngược lại, đường đi ngắn nhất trên đồ thị không chu trình là quy hoạch động theo thứ tự tô-pô. Cách nhìn này có giá trị thực dụng: khi quy hoạch động của bạn có chu trình phụ thuộc, hãy chuyển sang Dijkstra trên đồ thị trạng thái thay vì cố ép một thứ tự tính.

- A5. Bằng nhảy nhị phân: tiền xử lý bảng $up[v][k]$ — tổ tiên thứ 2^k của v — trong $O(n \log n)$, rồi mỗi truy vấn nâng hai đỉnh về cùng độ sâu và nhảy đồng thời từ bước lớn xuống bước nhỏ, tốn $O(\log n)$. Cách thứ hai là chuyển sang dãy Euler và biến bài toán thành cực tiểu trên đoạn $(B/10)$, sau đó dùng sparse table cho truy vấn $O(1)$ — lựa chọn tốt khi số truy vấn rất lớn.

NÓI LẠI THẬT ĐƠN GIẢN

Đồ thị chỉ là cách vẽ lại một bài toán thành các điểm và các đường nối. Điều đáng nói không phải bản thân hình vẽ mà là việc rất nhiều bài toán khác nhau, sau khi vẽ lại, hoá ra là cùng một bài.

Hãy nghĩ tới bản đồ tàu điện. Các ga là điểm, các tuyến là đường nối. Câu hỏi “đi từ ga A tới ga B ít trạm nhất” được trả lời bằng cách loang dân: đánh dấu mọi ga cách A một trạm, rồi hai trạm, rồi ba — lần đầu chạm tới B là đáp án. Đó là duyệt rộng, và nó đúng vì bạn xét theo đúng thứ tự tăng dần của số trạm.

Nếu mỗi tuyến mất thời gian khác nhau thì cách loang đều không còn đúng, và bạn phải luôn đi tiếp từ ga có *thời gian tích lũy nhỏ nhất* trong số các ga đang chờ. Đó là Dijkstra. Điều kiện để nó đúng: đi thêm một tuyến không bao giờ làm giảm tổng thời gian. Nếu có tuyến “thường” làm giảm tổng thời gian — tức trọng số âm — thì cách này sai, và phải dùng cách chậm hơn nhưng chắc chắn: cứ cải thiện mọi tuyến nhiều lượt cho tới khi không cải thiện được nữa.

Còn mẹo mạnh nhất của chương này thì thế này: đỉnh *không nhất thiết là địa điểm*. Nếu bài toán nói bạn có hai cái bình 3 lít và 5 lít và cần đóng đúng 4 lít, thì hãy coi mỗi “tình huống” — ví dụ bình một có 2 lít, bình hai có 5 lít — là một điểm, và mỗi thao tác đổ là một đường nối. Bỗng nhiên bài đổ mẹo trở thành bài tìm đường trên một bản đồ vài chục điểm, và bạn giải nó bằng đúng cách loang ở trên.

Đó là toàn bộ ý của mô hình hoá, và nó cũng chính là điều Euler làm năm 1736 khi ông xoá bản đồ thành phố đi và chỉ giữ lại xem vùng đất nào nối với vùng đất nào.

Một câu: Khi bí, hãy hỏi “*cái gì là đỉnh, cái gì là cạnh*” — và nhớ rằng *đỉnh có thể là một trạng thái, không chỉ là một địa điểm*.

Chỗ không rút gọn được: Với đồ thị trạng thái, việc ước lượng số trạng thái phải làm bằng tay cho từng bài; không có cách nào tự động biết nó là 10^6 hay 10^{15} .

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ba dấu hiệu bài đồ thị	Quan hệ đôi một; ràng buộc thứ tự/phụ thuộc; và <i>trạng thái với bước chuyển</i> — dấu hiệu thứ ba đáng giá nhất.
••• Bảng chọn thuật toán đường đi ngắn nhất	Không trọng số: BFS. Trọng số 0/1: BFS 0–1 với deque. Không âm: Dijkstra. Có âm: Bellman–Ford. Mọi cặp, n nhỏ: Floyd–Warshall.
••• Chỗ Dijkstra dùng giả thiết không âm	Khi chốt đỉnh nhãn nhỏ nhất, nó tin rằng đường khác đi qua đỉnh chưa chốt chỉ có thể đắt hơn — đúng vì cạnh không làm giảm chi phí.
••• Bellman–Ford và chu trình âm	Thư giãn mọi cạnh $n - 1$ lần (đường ngắn nhất có $\leq n - 1$ cạnh); nếu vòng thứ n vẫn cải thiện thì có chu trình âm.
••• Duyệt sâu để lộ những gì	Thời điểm vào/ra cho: phát hiện chu trình, thứ tự tô-pô (ra giảm dần), cầu và khớp, thành phần liên thông mạnh.
••• Quy hoạch động và đường đi ngắn nhất	Là hai cách nói về cùng một thứ: đỉnh là trạng thái, cạnh là chuyển. Quy hoạch động có chu trình phụ thuộc $\Rightarrow$ dùng Dijkstra trên đồ thị trạng thái.
•• Ba mẹo mô hình hoá	Đồ thị trạng thái; đồ thị nhiều tầng (nhân bản theo số lần dùng khả năng đặc biệt); tách đỉnh (khi ràng buộc nằm ở đỉnh).
•• Duyệt rộng nhiều nguồn	Đẩy mọi nguồn vào hàng đợi lúc khởi tạo $\Rightarrow$ khoảng cách tới nguồn gần nhất trong một lượt $O(n + m)$.
•• Nhảy nhị phân	$up[v][k]$ = tổ tiên thứ 2^k ; tiền xử lý $O(n \log n)$, truy vấn tổ tiên chung $O(\log n)$; mở rộng được cho cực trị trên đường đi.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Dãy Euler	Biến bài tổ tiên chung thành bài cực tiểu trên đoạn $(B/10)$; phép quy dẫn đi được cả hai chiều.
●● A*	Dijkstra cộng ước lượng khoảng cách còn lại; tối ưu nếu ước lượng không bao giờ vượt quá thực tế.
●● Lỗi Floyd–Warshall phổ biến	Vòng k phải ở ngoài cùng; đảo thứ tự vòng lặp cho kết quả sai.
● Phân rã theo cạnh nặng	Chia cây thành các chuỗi để truy vấn trên đường đi quy về $O(\log n)$ truy vấn đoạn; mạnh nhưng thuộc loại tham khảo.
● Rào cản sắp xếp	Dijkstra chốt đỉnh theo đúng thứ tự khoảng cách nên thừa hưởng cận dưới của sắp xếp. Vượt được bằng cách đổi mô hình (trọng số nguyên, đồ thị vô hướng) hoặc bằng cách không sắp xếp đầy đủ — $O(m \log^{2/3} n)$, 2025.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Khi nào tôi nên dùng ma trận kề thay vì danh sách kề?

| Chỉ khi n nhỏ (vài nghìn trở xuống) và đồ thị dày, hoặc khi bạn cần hỏi “có cạnh giữa u và v không” rất nhiều lần trong $O(1)$. Ma trận tốn $O(n^2)$ bộ nhớ nên với $n = 10^5$ là không khả thi. Trong hầu hết bài thi đấu, danh sách kề là lựa chọn mặc định, và nếu cần kiểm tra cạnh nhanh thì dùng thêm một tập băm các cặp.

2. Dijkstra của tôi cho kết quả sai trên đồ thị có cạnh trọng số 0. Có phải vì trọng số 0?

| Không, trọng số 0 hoàn toàn hợp lệ với Dijkstra — giả thiết là *không âm*, và 0 không âm. Lỗi gần như chắc chắn nằm ở chỗ khác: quên bỏ qua bản ghi lỗi thời khi lấy ra khỏi hàng đợi ưu tiên, hoặc cập nhật nhãn nhưng không đẩy vào hàng đợi, hoặc so sánh sai khi hai nhãn bằng nhau. Nếu đồ thị chỉ có trọng số 0 và 1 thì hãy dùng duyệt rộng 0–1, vừa nhanh hơn vừa ít chỗ sai hơn.

3. Làm sao đếm số đường đi ngắn nhất chứ không chỉ độ dài?

| Thêm một mảng $\text{cnt}[v]$. Khi thư giãn cạnh (u, v) : nếu tìm được đường ngắn hơn thì $\text{cnt}[v] = \text{cnt}[u]$; nếu bằng đúng khoảng cách hiện tại thì $\text{cnt}[v] += \text{cnt}[u]$. Cách này hoạt động với cả duyệt rộng và Dijkstra, với điều kiện bạn xử lý mỗi đỉnh theo đúng thứ tự khoảng cách tăng dần. Nhớ lấy modulo nếu số đường đi lớn.

4. Bài của tôi có 10^6 đỉnh trạng thái nhưng tôi không muốn sinh trước toàn bộ đồ thị. Làm sao?

| Sinh cạnh *khi cần* — gọi là đồ thị ẩn. Thay vì lưu danh sách kề, viết một hàm “cho tôi các trạng thái kề của trạng thái này” và gọi nó trong vòng lặp duyệt. Cách này tiết kiệm bộ nhớ rất nhiều và là cách chuẩn cho các bài trò xếp hình hay bài đong nước. Điều cần thêm là một cấu trúc để đánh dấu trạng thái đã thăm — thường là bảng băm nếu trạng thái không ánh xạ gọn về số nguyên.

5. Thứ tự tô-pô có thể có nhiều kết quả. Làm sao lấy kết quả nhỏ nhất theo thứ tự từ điển?

| Dùng thuật toán Kahn nhưng thay hàng đợi bằng hàng đợi ưu tiên: luôn lấy ra đỉnh có chỉ số nhỏ nhất trong số các đỉnh không còn phụ thuộc. Chi phí thành $O(m \log n)$. Lưu ý đây là một câu hỏi khác với “thứ tự tô-pô bất kỳ”, và nhiều bài cổ tình hỏi bản khó hơn — hãy đọc kỹ đề.

6. Vì sao cùng một bài lại có bài giải bằng luồng cực đại, bài giải bằng Dijkstra?

| Vì hai họ bài toán trả lời hai câu hỏi khác nhau. Đường đi ngắn nhất hỏi về *một* đường; luồng hỏi về *nhiều* đường đồng thời với ràng buộc sức chứa. Dấu hiệu phân biệt trên đề: nếu có khái niệm “sức chứa”, “mỗi thứ chỉ dùng một lần”, “phân công” thì đó là luồng hoặc ghép đôi ($C/18$); nếu chỉ có chi phí đi lại thì đó là đường đi ngắn nhất.

7. Trong hệ thống thật, Dijkstra có đủ cho định tuyến bản đồ không?

| Không ở quy mô thật. Mạng đường bộ một quốc gia có hàng chục triệu đỉnh, và Dijkstra thuần tuý mất hàng giây cho một truy vấn — quá chậm cho dịch vụ trực tuyến. Các hệ thống thật dùng tiền xử lý nặng: phân cấp co rút, nhãn khoảng cách, hoặc phân vùng — những kỹ thuật đổi hàng giờ tiền xử lý lấy truy vấn dưới mili-giây. Nhưng tất cả đều xây trên Dijkstra và đều dùng đúng tính chất tham lam của nó, nên hiểu Dijkstra vẫn là điều kiện cần.

8. Phát hiện chu trình trong đồ thị vô hướng và có hướng khác nhau thế nào?

| Khác ở tiêu chí. Trong đồ thị vô hướng, gặp một đỉnh đã thăm mà không phải cha trực tiếp là có chu trình — hoặc đơn giản hơn, dùng DSU (B/11): cạnh nối hai đỉnh cùng nhóm tạo chu trình. Trong đồ thị có hướng, phải phân biệt “đã thăm xong” với “đang trên ngăn xếp”: chỉ cạnh trở về một đỉnh *đang trên ngăn xếp* mới tạo chu trình. Nhầm hai trạng thái này là một trong những lỗi phổ biến nhất khi cài.

I Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Đếm số thành phần liên thông	CSES “Counting Rooms”[6]	Duyệt sâu hoặc rộng trên lưới; chú ý độ sâu đệ quy với lưới lớn.
● Đường đi ngắn nhất trên lưới có vật cản	CSES “Labyrinth”[6]	Duyệt rộng kèm truy vết đường đi; bài mẫu cho việc lưu “đỉnh trước”.
● Kiểm tra đồ thị hai phía	CSES “Building Teams”[6]	Tô màu bằng duyệt rộng; so với cách DSU chặn lẻ ở B/11.
●● Đường đi ngắn nhất có trọng số	CSES “Shortest Routes I”[6]	Dijkstra với hàng đợi ưu tiên; nhớ dòng bỏ qua bản ghi lỗi thời.
●● Phát hiện chu trình âm	CSES “Cycle Finding”[6]	Bellman–Ford và truy vết chu trình; chú ý đỉnh không tới được từ nguồn.
●● Sắp thứ tự tô-pô và đường đi dài nhất trên đồ thị không chu trình	CSES “Course Schedule”, “Longest Flight Route”[6]	Quy hoạch động theo thứ tự tô-pô — ví dụ trực tiếp cho quan hệ ở mục 3.
●● Bài đong nước với hai bình	Bài kinh điển	Đồ thị trạng thái; ước lượng số trạng thái trước khi cài.
●● Số đường đi ngắn nhất giữa hai đỉnh	CSES “Investigation”[6]	Dijkstra kèm bốn đại lượng cùng lúc; luyện việc mở rộng thuật toán chuẩn.
●●● Thành phần liên thông mạnh và nén đồ thị	CSES “Planets and Kingdoms”[6]	Kosaraju hoặc Tarjan; sau khi nén thì chạy quy hoạch động trên đồ thị không chu trình.
●●● Tìm cầu và khớp	Bài kinh điển	Duyệt sâu với giá trị “lên cao nhất”; chú ý xử lý cạnh song song.
●●● Tổ tiên chung gần nhất với 10^5 truy vấn	CSES “Company Queries II”[6]	Nhảy nhị phân; rồi thử lại bằng dãy Euler cộng sparse table và so thời gian.
●●● Đường đi ngắn nhất khi được đi miễn phí tối đa k cạnh	Bài thi đấu kinh điển	Đồ thị nhiều tầng; đây là bài mẫu cho mẹo ở mục 5.
●●● A^* cho bài xếp hình 15 ô	Bài kinh điển	So số đỉnh phải xét giữa duyệt rộng, Dijkstra và A^* với hai hàm ước lượng khác nhau.

Kết chương và đọc tiếp

Đồ thị là chương của kỹ năng mô hình hoá, và ba câu hỏi ở khối *Cần nhớ* — đỉnh là gì, cạnh là gì, trọng số là gì — đáng thành phần xạ. Hai điều nữa đáng mang theo: đổi kho chờ thì đổi thuật toán, nên duyệt rộng, duyệt sâu và Dijkstra là ba biến thể của một khung; và quy hoạch động chính là đường đi ngắn nhất trên đồ thị trạng thái, nên hai chương này là hai cách nói về cùng một thứ.

Chương tiếp theo mở rộng mô hình đồ thị theo một hướng mới: thay vì hỏi về *một* đường đi, ta hỏi về *nhiều* đường đi đồng thời với ràng buộc sức chứa. Đó là luồng cực đại, và điều bất ngờ là rất nhiều bài không liên quan gì tới “dòng chảy” — phân công công việc, chọn tập đại diện, một số bài chia đôi tối ưu — đều quy về đúng mô hình đó (C/18).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Dijkstra, “A Note on Two Problems in Connexion with Graphs”	Numerische Mathematik, 1959	Ba trang, không có ký hiệu tiệm cận nào; đọc để thấy thuật toán được nghĩ ra như thế nào trước khi ngành có ngôn ngữ để nói về nó.
nền tảng	Duan, Mao, Mao, Shu, Yin, “Breaking the Sorting Barrier for Directed Single-Source Shortest Paths”	STOC, 2025 (Best Paper)	Đọc phần mở đầu: nó giải thích rào cản sắp xếp và ý tưởng tránh sắp xếp rõ hơn bất kỳ bài giới thiệu nào.
tra khi cần	Thorup, “Undirected Single-Source Shortest Paths with Positive Integer Weights in Linear Time”	Journal of the ACM, 1999	Ví dụ mẫu mực về việc đổi mô hình (số nguyên, vô hướng) để thoát một cận dưới.
tra khi cần	Tarjan, “Depth-First Search and Linear Graph Algorithms”	SIAM Journal on Computing, 1972	Bài gốc của thành phần liên thông mạnh; đọc kèm mục 2 của chương này.
tra khi cần	Fredman & Tarjan, “Fibonacci Heaps...”	Journal of the ACM, 1987	Nguồn của cận $O(m + n \log n)$; xem thêm phần đo thực nghiệm ở B/07 trước khi quyết định dùng.

Nguồn

- [1] R. E. Tarjan, “Depth-First Search and Linear Graph Algorithms,” *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.
- [2] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [3] M. Thorup, “Undirected Single-Source Shortest Paths with Positive Integer Weights in Linear Time,” *Journal of the ACM*, vol. 46, no. 3, pp. 362–394, 1999.
- [4] R. Duan, J. Mao, X. Mao, et al., “Breaking the Sorting Barrier for Directed Single-Source Shortest Paths,” in *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, Best Paper Award, 2025.
- [5] E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [6] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).